



5 UNSUPERVISED LEARNING

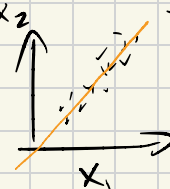
- no supervision signal for learning f as we had before
- we have a data training set (TS)
$$TS = \{x_i\}_{i=1}^N \quad (\text{no } y_i\text{'s})$$

• Why do this at all?

- ground truth labels are costly, but unlabeled data is easy to get
- in initial state of research project, we do not know what to look for (data mining)

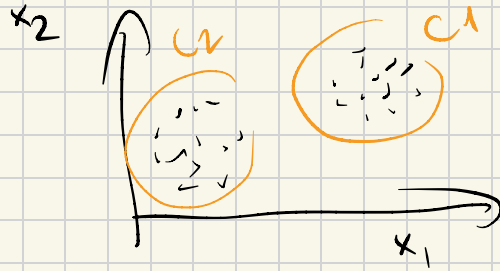
• Two core unsupervised tasks:

1) Dimensionality reduction: construct a mapping $\phi: \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$ with $D' < D$, yielding a compressed representation



2) Pattern discovery (mining, "clustering"),

find k clusters $C_1 \dots C_k$, maximizing intra-cluster similarity



5.1 Dimensionality Reduction

Motivate:

- curse of dimensionality: data may lie in a lower dimensional manifold
- Noise reduction (summarize correlated data, discarding small variance directions)
- Visualization: project to 2D/3D to inspect its structure

Two possibilities: $\tilde{x}_i = \phi(x_i)$ $\tilde{x}_i \in \mathbb{R}^{D'}$, $x_i \in \mathbb{R}^D$
 $D' < D$

1) Feature selection: $\tilde{x}_i$ is

a subset of x_i ; throw away uninformative features

— in the case of supervised learning,

LASSO

2) Generate new feature

• e.g. in linear case

$$\phi(x_i) = x_i V^T \quad V^T \in \mathbb{R}^{D \times D'}$$

• common algorithms:
linear

• SVD

• PCA

• ICA (Indep. comp. analysis)

• NMF (Nonneg. matrix fact.)

nonlinear:

• Kernel PCA

• local linear embedding $\rightarrow$ t-SNE

• Autoencoders

§.11 SINGULAR VALUE DECOMPOSITION (SVD)

- The SVD decomposes $X \in \mathbb{R}^{N \times D}$ as: $N \geq D$

$$X = U \Sigma V^T$$

where

- $U \in \mathbb{R}^{N \times N}$ orthogonal matrix, columns are the "left singular vectors"

- $\Sigma \in \mathbb{R}^{N \times N}$ rectangular matrix with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_D = 0$

- $V \in \mathbb{R}^{D \times D}$ orthogonal, columns are the "right, singular vectors"

$$X = \begin{bmatrix} | & | & | \\ x_1 & x_2 & \dots & x_D \\ | & | & | \end{bmatrix}^D = \begin{bmatrix} | & | & | \\ u_1 & u_2 & \dots & u_N \\ | & | & | \end{bmatrix}^N \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \\ & & & \sigma_D \\ & & & & 0 \end{bmatrix} \begin{bmatrix} | & | & | \\ v_1 & v_2 & \dots & v_D \\ | & | & | \end{bmatrix}^T$$

$\underbrace{\quad}_{t=1} \quad \dots \quad \underbrace{\quad}_T$

- Properties : $U^T U = U U^T = \mathbb{1}_N$

$$V^T V = V V^T = \mathbb{1}_D$$

- with this scaling, SVD guarantees a unique solution up to a minus sign

Using SVD for compression

$$X = \begin{bmatrix} 1 & & 1 \\ u_1 & \dots & u_m \\ 1 & & 1 \end{bmatrix} \begin{bmatrix} b_1 & & \\ & \ddots & \\ & & b_D \\ 0 & & \end{bmatrix} \begin{bmatrix} -v_1^T \\ \vdots \\ -v_D^T \end{bmatrix} = b_1 u_1 v_1^T + b_2 u_2 v_2^T + \dots + b_D u_D v_D^T + 0$$

- u_i, v_i^T = rank one matrices

- $u_i, i > D$ are multiplied by 0

$\Rightarrow$ Rewrite SVD as "economy SVD":

$$X = \hat{U} \hat{\Sigma} \hat{V}^T$$

- $\hat{U} = U_{1:N, 1:D}$

- $\hat{\Sigma} = \Sigma_{1:D, 1:D}$

- $\hat{V} = V_{1:D, 1:D}$

$$\Rightarrow U \Sigma V^T = \hat{U} \hat{\Sigma} \hat{V}^T$$

compress data by:

$$X = b_1 u_1 v_1^T + b_2 u_2 v_2^T + \dots + b_R u_R v_R^T + \text{red part}$$

$+ b_{R+1} u_{R+1} v_{R+1}^T + \dots + b_D u_D v_D^T$

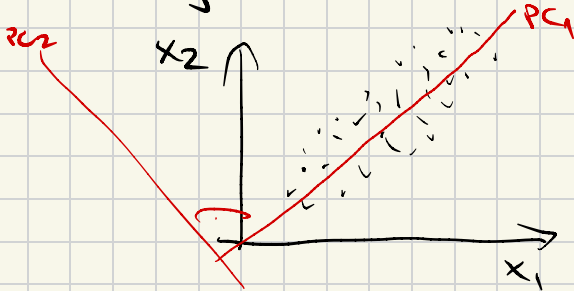
DROP RED PART:

$$X \approx \sum_{i=1}^{R, R < D} b_i u_i v_i^T \approx \hat{U} \hat{\Sigma} \hat{V}^T$$

- Eckart-Young theorem, 1936: LET $X \in \mathbb{R}^{N \times D}$ have SVD $X = U \Sigma V^T$. Then the best rank R approximation $\hat{X}$ minimizing the Frobenius norm $\hat{X} = \arg \min_{\text{rank}(Y)=R} \|X - Y\|_F$ is given by $\hat{X} = \hat{U}_R \hat{\Sigma}_R \hat{V}_R^T$.
- How to choose R ?
depends on your research question

5.1.2 PRINCIPAL COMPONENT ANALYSIS (PCA)

- Idea: rotate the system such that (orthogonal) axes consecutively align with directions of highest variance in the data space



- OPTIMIZATION CRITERION:

$$\begin{aligned} v^* &= \underset{v, \|v\|=1}{\operatorname{argmax}} \operatorname{VAR}(Xv) \\ &= \underset{v, \|v\|=1}{\operatorname{argmax}} v^T \operatorname{Cov}(X) v \\ &= \underset{v, \|v\|=1}{\operatorname{argmax}} v^T \underbrace{(X - \underbrace{\mathbb{1}\bar{x}}_{\text{col. mean}})(X - \mathbb{1}\bar{x})}_{=: S} v \\ &= \underset{v}{\operatorname{argmax}} \left[v^T S v - \lambda (v^T v - 1) \right] \end{aligned}$$

↪ regularization parameter

- Differentiate and set to 0:

$$\begin{aligned} \nabla_v (v^T S v - \lambda (v^T v - 1)) &= 2Sv - 2\lambda v \\ \Rightarrow Sv &= \lambda v \end{aligned}$$

$\Rightarrow$ eigenvalue eqn with $\lambda = \text{eigenvalue}$

PCA vs SVD :

$$\begin{aligned} X^T X &= (U \Sigma V^T)^T (U \Sigma V^T) \\ &= V \Sigma^2 V^T \end{aligned}$$

$$\begin{aligned} X^T X - V &= V \Sigma^2 \overset{=1}{V^T V} \\ &= V \Sigma^2 \end{aligned}$$

$\Rightarrow$ eigenvalue equation; same as in PCA

$\Rightarrow$ the interpretation of the right singular vectors is that these are the eigenvectors of the cov. matrix $(X^T X)$

Relation to explained variation

• say we have $X = U \Sigma V^T$ $\Sigma = \text{diag}(b_1, \dots, b_D)$,

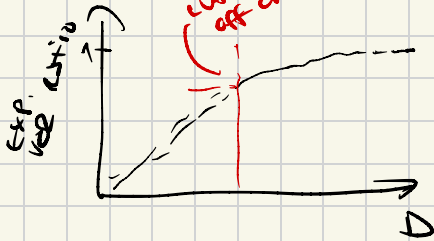
$b_1 > b_2 > \dots > b_D$, then

$b_i^2 = \text{variance along the } i\text{-th PC}$

- Total variance explained in the data:

$$\text{tot. var} = \sum_{i=1}^R b_i^2$$

- Exp. variation ratio: $\frac{\sum_{i=1}^R b_i^2}{\sum_{i=1}^D b_i^2}$



PCA Algorithm for compression

1. Data $X \in \mathbb{R}^{n \times D}$
2. Mean-center $X - \mathbf{1}\bar{x} = \tilde{X}$
3. compute cov. matrix

$$S = \tilde{X}^T \tilde{X}$$

unbiased est
 $\frac{1}{N-1} \tilde{X}^T \tilde{X}$

4. eigenvalue decomposition

$$S V = V \Lambda$$

V = eigenvectors Λ = eigenvalues

5. choose number of components

select R

6. Project data to obtain PCs

$$\begin{array}{l} \text{PCs} \\ \text{or "scores"} \end{array} \quad \begin{array}{l} \text{"loading"} \\ z = \tilde{X} V_{:,1:r} \end{array} \Rightarrow z \in \mathbb{R}^{N \times r}$$

$\underbrace{\quad}_{\in \mathbb{R}^{D \times r}}$

$\Rightarrow$ IMPORTANT: STANDARDIZE DATA
IF YOU WANT EACH DIMENSION TO
EQUALLY CONTRIBUTE

5.1.3 Autoencoders (AE)

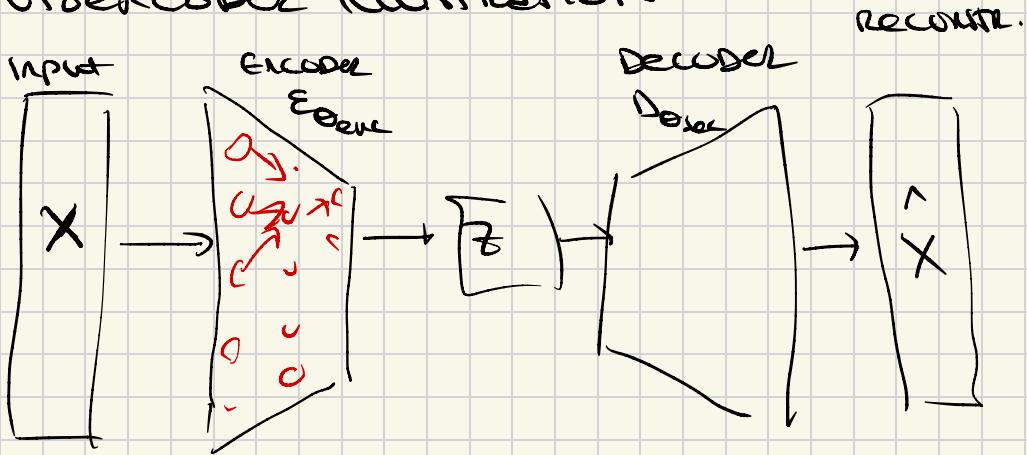
PCA as an AE

• "ENCODER" $z = \tilde{X} V$
 $z_i = V \tilde{x}_i$

• "DECODER" $\hat{\tilde{x}}_i = V^T z_i$
"reconstruction"

$$X \xrightarrow{V} \text{Encoder} \rightarrow z \xrightarrow{V^T} \text{Decoder} \rightarrow \hat{X}$$

Autoencoder illustration



→ PCA can be seen as a special case of a linear AE

LINEAR AE

• ENCODER $z_i = x_i W_e + b_e$

• DECODER: $\hat{x}_i = z_i W_d + b_d$

OPTIMIZATION OBJECTIVE:

• minimize the reconstruction error

$$\text{minimize } \mathcal{L}(\theta_{enc}, \theta_{dec}) = \sum_i \|x_i - \hat{x}_i\|^2$$

- for a linear AE

$$\hat{x}_i = x_i W_2 W_1$$

- It's provable that minimizing this loss for linear AEs amounts to selecting same subspace as in PCA (W matrices)

- For nonlinear problems, the obj. becomes

$$\min \sum_i \|x_i - (D_{\theta_1} \circ E_{\theta_2}) x_i\|^2$$

where D and E could be parametrized neural networks

- typically we add regularization:

$$\mathcal{K}(\theta_1, \theta_2) = \sum_{i=1}^N \|x_i - D \circ E \cdot x_i\|^2 + \tau \mathcal{R}(\theta_1, \theta_2)$$

adding regularization

- Specific Variants of AEs :

1) Denoising AE : Trained to reconstruct clean input from corrupted version

2) Sparse AE : Encourage sparsity in latent representation (via ℓ_1 penalty)

$$R(\theta_d, \theta_e) = \tau \sum_{i=1}^N \|E(x_i)\|_1$$

(connection to LASSO regression)

3) Variational AEs : probabilistic latent representation (z = random variable)

- Use gradient based optimization

5.2 clustering

- Idea: Identify groups of similar instances

$$x_i \approx x_{i'}$$

based on some notion of similarity or distance metric, and assign them to cluster.

- Assumption: Given a dataset $\{x_i\}_{i=1:N}$ we assume that (unknown) latent labels y_i exist.

- Two basic fits:

1) reduce complexity by analyzing one cluster at a time

For algorithms with complexity $O(N^P)$, $P > 1$, clustering can reduce complexity

cost:

$$\sum_{k=1}^K O(N_k^P) < O(N^P)$$

Example: if we want to perform pairwise similarity comparisons, we have $O(N^2) \Rightarrow$

comparing all-pairs only within clusters reduces complexity

2) often may reveal a simpler underlying structure improving human interpretability

we distinguish between two types of tasks:

- Task 1: Cluster Assignment: Determine which data point belongs to the same cluster
- Task 2: Cluster Representation: summarize clusters with a representative data point

(a) Cluster mean $C_k = \text{cluster } k$

$$\bar{x}_k = \frac{1}{N_k} \sum_{i \in C_k} x_i = \underset{x \in \mathbb{R}^D}{\operatorname{argmin}} \sum_{i \in C_k} \|x_i - x\|_2^2$$

(b) Cluster median

$$\operatorname{median}(C_k) = \underset{x \in \mathbb{R}^D}{\operatorname{argmin}} \|x_i - x\|_1$$

→ no closed form solution for $D > 1$

(c) Medoid:

$$\operatorname{medoid}(C_k) = \underset{x \in \{x_i: i \in C_k\}}{\operatorname{argmin}} \sum_{j \in C_k} d(x_j, x)$$

→ representative is in your cluster

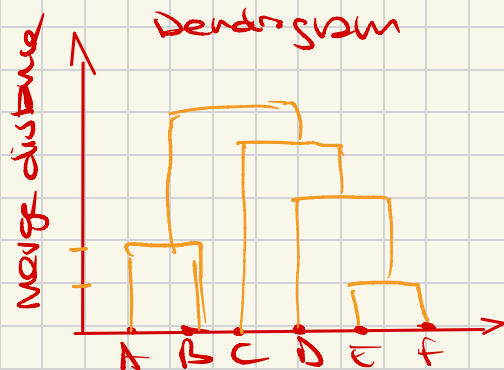
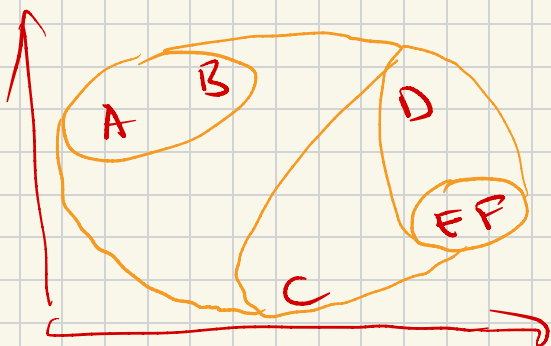
→ it can be general, but oftentimes taken as

$$\arg \min_{x \in \{x_i \in C_k\}} \sum_{j \in C_k} \|x_j - x\|_1$$

• next, look at how to perform task 1

5.2.1 Hierarchical Clustering

- Initialization: Assign each data point its own cluster $C_k = \{x_k\}$ for $k=1 \dots N$
- loop (bottom up)
 1. Repeat up to $N-1$ iterations:
 - identify two closest clusters according to a "linkage criterion"
 - merge into new cluster



- stopping criterion: Stop early to produce $K < N$ clusters, e.g. when:
 - a fixed number of clusters K is reached ($K = \text{hyperparameter}$)
 - the next merge distance exceeds some threshold ϵ ($\epsilon = \text{hyperparameter}$)
- what is "closest": we define inter-cluster distance via a linkage criterion:
 - 1) single linkage criterion between any two members:

$$d(C_k, C_{k'}) = \min_{i \in C_k, j \in C_{k'}} d(x_i, x_j)$$

→ look at every pair of points, one from C_k , one from $C_{k'}$, find smallest distance of two